CFX DSP Instruction Set
Quick Reference

ON Semiconductor®

M-20550-004

Copyright © 2009 Semiconductor Components Industries, LLC

Data Movement

CLR	X[single-mode(X)]	dst = 0	Ⓜ
LOAD	reg, X[imm16u]	dst = src	Ⓛ
	xmem-reg, X[single-mode(X)]		Ⓜ
	reg, Y[imm16u]		Ⓛ
	by-reg, Y[single-mode(Y)]		Ⓜ
	reg, imm20s		Ⓛ
	d-reg, imm24s		Ⓛ
	d-reg, imm5s		Ⓜ
	d-reg, imm6s		Ⓐ
	l-reg, imm24s		Ⓛ
	xstep-reg, imm5s		Ⓜ
	ystep-reg, imm5s		Ⓜ
LOAD.DW	ab-reg, imm24s		Ⓛ
	ab-reg, imm5s		Ⓜ
	d-reg, imm6s		Ⓐ
MOVE	reg, reg		Ⓜ
	d-reg, d-reg[1]		Ⓐ
	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond, dst = src	Ⓐ
MOVE.DW	ab-reg, ab-reg	dst = src	Ⓜ
MOVE.EX	ab-reg, ab-reg		Ⓜ
MOVE.NX	d-reg, d-reg		Ⓐ
STORE	X[imm16u], reg		Ⓛ
	X[single-mode(X)], xmem-reg		Ⓜ
	Y[imm16u], reg		Ⓛ
	Y[single-mode(Y)], by-reg		Ⓜ
	X[imm16u], imm8s		Ⓛ
	Y[imm16u], imm8s		Ⓛ
SWAP	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond, swap	Ⓐ
SWAP registers cannot be from same register file (A, B, X, Y)			

Indirect Addressing Modes

single-mode(X|Y)		dual-mode(X|Y) ‡
x|yp-reg	x|yp-reg—	x|yp-reg
x|yp-reg++	x|yp-reg += 2	x|yp-reg++
x|yp-reg += step †	x|yp-reg –= 2	x|yp-reg += step †
x|yp-reg +%= step †	x|yp-reg –= step †	x|yp-reg +%= step †
synonyms[1]		
x|yp-reg +	x|yp-reg -	x|yp-reg +%!
x|yp-reg += 1	x|yp-reg -= 1	
x|yp-reg +!	x|yp-reg -!	

† step must match pointer. e.g. xp5 += xstep5
‡ not XP4-7

Instruction Words

Ⓛ	Long
Ⓐ ||| Ⓜ	Arithmetic-Move
Ⓜ >>> Ⓜ	Move-Move

Dual Data Movement: x-side !! y-side Ⓜ

X-side Instruction		
LOAD	ax-reg, X[dual-mode(X)]	dst = src
	ax-reg, imm2s	
MOVE	ab-reg, x-reg	
	xy-reg, a-reg	
STORE	X[dual-mode(X)], a-reg	
Y-side Instruction		
LOAD	by-reg, Y[dual-mode(Y)]	dst = src
	by-reg, imm2s	
MOVE	ab-reg, y-reg	
	xy-reg, b-reg	
STORE	Y[dual-mode(Y)], b-reg	

P Data Movement

LOAD	reg, P[imm16u]	dst = src	Ⓛ
	reg, P[single-mode(X)]		Ⓛ
	reg, P[single-mode(Y)]		Ⓛ
LOAD.EX	ab-reg, P[imm16u]		Ⓛ
	ab-reg, P[single-mode(X)]		Ⓛ
	ab-reg, P[single-mode(Y)]		Ⓜ
STORE	P[imm16u], reg		Ⓛ
	P[single-mode(X)], reg		Ⓛ
	P[single-mode(Y)], reg		Ⓛ
	P[imm16u], imm8s		Ⓛ
STORE.EX	P[imm16u], ab-reg		Ⓛ
	P[single-mode(X)], ab-reg		Ⓛ
	P[single-mode(Y)], ab-reg		Ⓜ

XY Data Movement

LOAD.DW	ab-reg, XY[imm16u]	dst = src	Ⓛ
	ab-reg, XY[single-mode(X)]		Ⓜ
STORE.DW	XY[imm16u], ab-reg		Ⓛ
	XY[single-mode(X)], ab-reg		Ⓜ
	XY[imm16u], imm8s		Ⓛ

cond3

[X_]LT	[X_]LE	[X_]GE	[X_]GT
[X_]LTZ[1]	[X_]LEZ[1]	[X_]GEZ[1]	[X_]GTZ[1]
[X_]EQ	[X_]ZERO	[X_]NE	[X_]NOT_ZERO
[X_]CARRY	[X_]NO_CARRY		

Use !*condition* to select logical complement of *condition*[1]

Data Movement

Stack Manipulation

ADD	SP, imm16	sp = sp + src	Ⓛ
	SP, imm7s		Ⓜ
	xp-reg, SP, imm16	dst = sp + src	Ⓛ
	xp-reg, SP, imm6n		Ⓜ
LOAD	reg, X[SP + imm16]	dst = src	Ⓛ
	stack-reg6, X[SP + imm6n]		Ⓜ
	stack-reg4, X[SP + imm4n]		Ⓜ
STORE	X[SP + imm16], reg		Ⓛ
	X[SP + imm6n], stack-reg6		Ⓜ
	X[SP + imm4n], stack-reg4		Ⓜ
SUB	SP, imm16[1]	sp = sp - src	Ⓛ
	SP, imm7sn[1]		Ⓜ
	xp-reg, SP, imm16[1]	dst = sp - src	Ⓛ
	xp-reg, SP, imm6p[1]		Ⓜ
stack-reg6	d-reg xp-reg l-reg h-reg e-reg LR ILPC SR ILSR		
stack-reg4	yp-reg LC LSA LEA SP		

Pointer Arithmetic

ADD	xp-reg, xp-reg, xstep-reg	dst = src1 + src2	Ⓜ
	xp-reg, xp-reg, imm3s		Ⓜ
	yp-reg, yp-reg, ystep-reg		Ⓜ
	yp-reg, yp-reg, imm3s		Ⓜ
	xp-reg, xp-reg, imm16		Ⓛ
SUB	xp-reg, xp-reg, imm3sn[1]	dst = src1 - src2	Ⓜ
	yp-reg, yp-reg, imm3sn[1]		Ⓜ
	xp-reg, xp-reg, imm16[1]		Ⓛ
SUBSTEP	xp-reg	dst = dst – step	Ⓜ
	yp-reg	step chosen by dst	Ⓜ

Registers

a-reg	A0 A1	b-reg	B0 B1
x-reg	X0 X1	y-reg	Y0 Y1
ab-reg	A0 A1 B0 B1	xy-reg	X0 X1 Y0 Y1
ax-reg	A0 A1 X0 X1	by-reg	B0 B1 Y0 Y1
d-reg	A0 A1 B0 B1 X0 X1 Y0 Y1		
l-reg	A0.L A1.L B0.L B1.L	h-reg	A0.H A1.H B0.H B1.H
e-reg	A0.E A1.E B0.E B1.E		
xp-reg	XP0-7	xstep-reg	XSTEP0-7
yp-reg	YP0-3	ystep-reg	YSTEP0-3
reg	d-reg xp-reg yp-reg xstep-reg ystep-reg l-reg h-reg e-reg XCIRC0-7 YCIRC0-3 LR ILPC SR ILSR LC LSA LEA SP		
xmem-reg	d-reg xp-reg yp-reg l-reg h-reg e-reg		

[1] This instruction form, condition or addressing mode requires version 3 or greater of the CFX assembler.

CFX DSP Instruction Set
Quick Reference

ON Semiconductor®

M-20550-004
Copyright © 2009 Semiconductor Components Industries, LLC

Instruction Words

Ⓛ	Long
Ⓐ ||| Ⓜ	Arithmetic-Move
Ⓜ >>> Ⓜ	Move-Move

ARITHMETIC

Arithmetic Ⓐ

ABS	d-reg, d-reg	dst = |src|
ADD	d-reg, d-reg, d-reg	dst = src1 + src2
	d-reg, d-reg, imm3s	
	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond dst = dst + src
	d-reg, imm3s, cond3 d-reg, imm3s if cond3[1]	
ADD.DW	d-reg, d-reg, imm3s	dst = src1 + src2
	d-reg, imm3s, cond3 d-reg, imm3s if cond3[1]	if cond, dst = dst + src
ADD.FR	d-reg, d-reg, imm3s	dst = src1 + fract(src2)
ADD.NX	d-reg, d-reg, d-reg	dst = src1 + src2
	d-reg, d-reg, imm3s	
ADD.NX.DW	d-reg, d-reg, imm3s	
ADDSH	d-reg, d-reg, d-reg	dst = (src1 + src2) >> 1
CMP	d-reg, d-reg	src1 – src2
	d-reg, imm3s	
CMP.DW	d-reg, imm3s	
CMPU.NX	d-reg, d-reg	
	d-reg, imm3s	
CMPU.NX.DW	d-reg, imm3s	
DIVST	d-reg, d-reg, d-reg	Division Step
LOG2ABS	d-reg, d-reg	dst = LOG2(|src|)
NLOG2ABS	d-reg, d-reg	dst = –LOG2(|src|)
MAX	d-reg, d-reg, d-reg	dst = MAX(src1, src2)
MIN	d-reg, d-reg, d-reg	dst = MIN(src1, src2)
NEG	d-reg, d-reg	dst = –src
	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond, dst = –src
SUB	d-reg, d-reg, d-reg	dst = src1 – src2
	d-reg, d-reg, imm3sn[1]	
	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond dst = dst – src
	d-reg, imm3sn, cond3[1] d-reg, imm3sn if cond3[1]	
SUB.DW	d-reg, d-reg, imm3sn[1]	dst = src1 - src2
	d-reg, imm3sn, cond3[1] d-reg, imm3sn if cond3[1]	if cond, dst = dst - src
SUB.FR	d-reg, d-reg, imm3s[1]	dst = src1 - fract(src2)
SUB.NX	d-reg, d-reg, d-reg	dst = src1 – src2
	d-reg, d-reg, imm3sn[1]	
SUB.NX.DW	d-reg, d-reg, imm3sn[1]	
SUBSH	d-reg, d-reg, d-reg	dst = (src1 – src2) >> 1

Dual Arithmetic: x-side !! y-side Ⓐ

X-side Instruction		
ADD	a-reg, a-reg, ab-reg	dst = src1 + src2
ADDSH		dst = (src1 + src2) >> 1
SUB		dst = src1 – src2
SUBSH		dst = (src1 – src2) >> 1
MUL	a-reg, x-reg, xy-reg	dst = src1 × src2
MULADD		dst = dst + (src1 × src2)
MULNEG		dst = –(src1 × src2)
MULSUB		dst = dst – (src1 × src2)
NOP can only be used if Y-side instruction is MUL*		
NOP		no operation
All below can only be used if Y-side instruction is not MUL*		
ADD.NX	a-reg, a-reg, ab-reg	dst = src1 + src2
SUB.NX		dst = src1 – src2
CMP	a-reg, ab-reg	src1 – src2
CMPU.NX		
MAX	a-reg, a-reg, ab-reg	dst = MAX(src1, src2)
MIN		dst = MIN(src1, src2)
AND		dst = src1 AND src2
OR		dst = src1 OR src2
ABS	a-reg, ab-reg	dst = |src|
NEG		dst = –src
NLOG2ABS		dst = –LOG2(|src|)
MOVE.NX		dst = src
Y-side Instruction		
ADD	b-reg, b-reg, ab-reg	dst = src1 + src2
ADDSH		dst = (src1 + src2) >> 1
SUB		dst = src1 – src2
SUBSH		dst = (src1 – src2) >> 1
MUL	b-reg, y-reg, xy-reg	dst = src1 × src2
MULADD		dst = dst + (src1 × src2)
MULNEG		dst = –(src1 × src2)
MULSUB		dst = dst – (src1 × src2)

Multiply Ⓐ

ADDMUL	ab-reg, xy-reg, xy-reg, xy-reg	product = src1 × (src2 + src3)
ADDMUL*acc*		
SUBMUL		product = src1 × (src2 – src3)
SUBMUL*acc*		
MUL[*.ss*]	ab-reg, xy-reg, xy-reg	product = src1 × src2
MUL*acc*[*.ss*]		
Accumulation Mode		
acc	*none*	dst = product
	ADD	dst = dst + product
	NEG	dst = –product
	SUB	dst = dst – product
Signed/Unsigned Mode		
.ss	*none*	signed × signed
	.SU	signed × unsigned
	.UU	unsigned × unsigned
The following combinations of src1, src2, src3 are allowed in ADDMUL* and SUBMUL*:		
x-reg, x-reg, y-reg	x-reg, y-reg, y-reg	
y-reg, x-reg, y-reg	y-reg, x-reg, x-reg	

Registers

a-reg	A0 A1	b-reg	B0 B1
x-reg	X0 X1	y-reg	Y0 Y1
ab-reg	A0 A1 B0 B1	xy-reg	X0 X1 Y0 Y1
ax-reg	A0 A1 X0 X1	by-reg	B0 B1 Y0 Y1
d-reg	A0 A1 B0 B1 X0 X1 Y0 Y1		
l-reg	A0.L A1.L B0.L B1.L	h-reg	A0.H A1.H B0.H B1.H
e-reg	A0.E A1.E B0.E B1.E		
xp-reg	XP0-7	xstep-reg	XSTEP0-7
yp-reg	YP0-3	ystep-reg	YSTEP0-3
reg	d-reg xp-reg yp-reg xstep-reg ystep-reg l-reg h-reg e-reg XCIRC0-7 YCIRC0-3 LR ILPC SR ILSR LC LSA LEA SP		
xmem-reg	d-reg xp-reg yp-reg l-reg h-reg e-reg		

cond3

[X_]LT	[X_]LE	[X_]GE	[X_]GT
[X_]LTZ[1]	[X_]LEZ[1]	[X_]GEZ[1]	[X_]GTZ[1]
[X_]EQ	[X_]ZERO	[X_]NE	[X_]NOT_ZERO
[X_]CARRY	[X_]NO_CARRY		

Use !*condition* to select logical complement of *condition*[1]

[1] This instruction form, condition or addressing mode requires version 3 or greater of the CFX assembler.

CFX DSP Instruction Set
Quick Reference

ON Semiconductor®

M-20550-004
Copyright © 2009 Semiconductor Components Industries, LLC

Logic Ⓐ

AND	d-reg, d-reg, d-reg	dst = src1 AND src2
	d-reg, d-reg, imm3s	
AND.DW	d-reg, d-reg, imm3s	
NOT	d-reg, d-reg[1]	dst = NOT src
OR	d-reg, d-reg, d-reg	dst = src1 OR src2
	d-reg, d-reg, imm3s	
OR.DW	d-reg, d-reg, imm3s	
XOR	d-reg, d-reg, d-reg	dst = src1 XOR src2
	d-reg, d-reg, imm3s	
XOR.DW	d-reg, d-reg, imm3s	
BITCLR	d-reg, bit-index	Clear bit in dst
BITSET	d-reg, bit-index	Set bit in dst
BITTGL	d-reg, bit-index	Toggle bit in dst
BITTST	d-reg, bit-index	Test bit in dst

Shift Ⓐ

SHLL	d-reg, d-reg, d-reg	dst = src1 << src2
	d-reg, d-reg, imm3u	
	d-reg, imm6u	dst = dst << src
	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond dst = dst << src
	d-reg, imm3u, cond3 d-reg, imm3u if cond3[1]	
SHLL.NX	d-reg, d-reg, d-reg	dst = src1 << src2
	d-reg, d-reg, imm3u	
	d-reg, imm6u	dst = dst << src
SHRA	d-reg, d-reg, d-reg	dst = src1 >> src2
	d-reg, d-reg, imm3u	
	d-reg, imm6u	dst = dst >> src
	d-reg, d-reg, cond3 d-reg, d-reg if cond3[1]	if cond dst = dst >> src
	d-reg, imm3u, cond3 d-reg, imm3u if cond3[1]	
SHRL.NX	d-reg, d-reg, d-reg	dst = src1 >> src2
	d-reg, d-reg, imm3u	
	d-reg, imm6u	dst = dst >> src

Instruction Words

Ⓛ	Long
Ⓐ ||| Ⓜ	Arithmetic-Move
Ⓜ >>> Ⓜ	Move-Move

Logic and Control Flow

Control Flow Instructions Ⓛ

CALL[.1D]	imm16u	lr = return address pc = src
CALL[.2D]	xp-reg	
GOTO[.0D|.1D]	imm16u	pc = src
GOTO[.2D]	xp-reg	
GOTO[.0D|.2D]	imm16u, cond5 imm16u if cond5[1]	if cond pc = src
GOTO[.2D]	xp-reg, cond5 xp-reg if cond5[1]	
INTERRUPT	imm5u	software interrupt
RETURN[.2D]		pc = lr
RETURNI[.0D|.2D]		pc = ilpc, sr = ilsr

Hardware Loops Ⓛ

LOOP[.2D]	imm12u, imm16u	begin hardware loop
LOOP[.2D]	d-reg, imm16u	
ENDLOOP		pop loop stack

Status Register Manipulation Ⓛ

BITCLR	SR, imm4u	clear bit in sr
BITSET	SR, imm4u	set bit in sr

Status Register Bits

0	Rounding Mode	8	Y Negative Flag
1	Bias Mode	9	Y Zero Flag
2	Interrupt Enable	10	Y Overflow Flag
3	X Carry Flag	11	Saturation Mode
4	X Negative Flag	12	X Extension In Use Flag
5	X Zero Flag	13	Y Extension In Use Flag
6	X Overflow Flag	14	Saturated Flag
7	Y Carry Flag	15	Flag Updates Mode

Misc

NOP		no operation Ⓛ
NOP		no operation Ⓜ
NOP		no operation Ⓐ
BREAKPOINT		software breakpoint Ⓛ
SLEEP		sleep Ⓛ
SLEEP	imm16u	sleep, resume at src Ⓛ

cond5

[X_|Y_]LT	[X_|Y_]LE	[X_|Y_]GE	[X_|Y_]GT
[X_|Y_]LTZ[1]	[X_|Y_]LEZ[1]	[X_|Y_]GEZ[1]	[X_|Y_]GTZ[1]
[X_|Y_]EQ	[X_|Y_]ZERO	[X_|Y_]NE	[X_|Y_]NOT_ZERO
FALSE	0[1]	TRUE	1[1]
SAT	NO_SAT	[X_|Y_]CARRY	[X_|Y_]NO_CARRY
FULL	NOT_FULL	[X_|Y_]EXTEN	[X_|Y_]NO_EXTEN
EMPTY	NOT_EMPTY	[X_|Y_]OFLOW	[X_|Y_]NO_OFLOW

Use !*condition* to select logical complement of *condition*[1]

cond3

[X_]LT	[X_]LE	[X_]GE	[X_]GT
[X_]LTZ[1]	[X_]LEZ[1]	[X_]GEZ[1]	[X_]GTZ[1]
[X_]EQ	[X_]ZERO	[X_]NE	[X_]NOT_ZERO
[X_]CARRY	[X_]NO_CARRY		

Use !*condition* to select logical complement of *condition*[1]

Registers

a-reg	A0 A1	b-reg	B0 B1
x-reg	X0 X1	y-reg	Y0 Y1
ab-reg	A0 A1 B0 B1	xy-reg	X0 X1 Y0 Y1
ax-reg	A0 A1 X0 X1	by-reg	B0 B1 Y0 Y1
d-reg	A0 A1 B0 B1 X0 X1 Y0 Y1		
l-reg	A0.L A1.L B0.L B1.L	h-reg	A0.H A1.H B0.H B1.H
e-reg	A0.E A1.E B0.E B1.E		
xp-reg	XP0-7	xstep-reg	XSTEP0-7
yp-reg	YP0-3	ystep-reg	YSTEP0-3
reg	d-reg xp-reg yp-reg xstep-reg ystep-reg l-reg h-reg e-reg XCIRC0-7 YCIRC0-3 LR ILPC SR ILSR LC LSA LEA SP		
xmem-reg	d-reg xp-reg yp-reg l-reg h-reg e-reg		

[1] This instruction form, condition or addressing mode requires version 3 or greater of the CFX assembler.